COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010**	**SECTION:**	CPE21S4
DATE PERFORMED:	07/ 29 / 26	**DATE SUBMITTED:**	07 / 29 / 26
NAME:	Mangcucang, Rhianne L.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 4.1 **Array-Based vs. STL Stack Implementations**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

```
#include <iostream>
#include <stack>


int main (){
    //create an object:
    std::stack<char> stack1;

    std::cout<<"Is the stack empty?" <<stack1.empty() << std::endl;
    stack1.push('r');
    std::cout<<"Is the stack empty?" <<stack1.empty() << std::endl;
    std::cout<<"Current top of the stack:"<<stack1.top()<<std::endl;
    stack1.push('h');
    std::cout<<"Current top of the stack:"<<stack1.top()<<std::endl;
    stack1.push('i');
    std::cout<<"Current top of the stack:"<<stack1.top()<<std::endl;
    std::cout<<"Is the stack empty?" <<stack1.empty() << std::endl;
    std::cout<<"What is the size of the stack?" <<stack1.size() << std::endl;
```

```
    //swap
    std::stack<char> stack2;
    stack2.push('D');

    stack1.swap(stack2);

    std::cout << "Top after swap: " << stack1.top() << std::endl;

    //emplace
    stack1.emplace('K');

    std::cout << "Top after emplace: " << stack1.top() << std::endl;
    std::cout << "size: " << stack1.size() << std::endl;


    return 0;
}
```

COMPUTER ENGINEERING

QUEZON CITY

CODE ANALYSIS

The C++ Standard Template Library (STL) stack (std::stack) with character (char) values. First, it includes <iostream> for printing outputs and <stack> to use the built-in stack features. Inside main(), an empty stack called stack1 is created. Calling stack1.empty() at the start gives 1 (true) because the stack does not have any items yet.

Next, the code adds items to stack1 using push(). It pushes 'r', 'h', and 'i' one after another. Since stacks follow the Last-In, First-Out (LIFO) rule, the item added last becomes the current top. Checking stack1.top() shows 'r', then 'h', and finally 'i'. After adding these three items, stack1.empty() returns 0 (false), and stack1.size() returns 3.

The code then creates a second stack named stack2 containing the letter 'D'. By calling stack1.swap(stack2), the contents of the two stacks are swapped. Now, stack1 only has 'D', while stack2 gets the three items from earlier. Printing stack1.top() right after the swap shows 'D'.

Lastly, the program uses stack1.emplace('K') to add 'K' directly onto stack1. Unlike push(), emplace() creates the item directly inside the stack's memory. After this step, stack1.top() shows 'K', and stack1.size() gives a final count of 2. The program then ends cleanly with return 0;.

ILO B1

```cpp
#include<iostream>

//global variable:
#define maxSize 10
int stackArr[maxSize];
int top = -1, newData;
//function prototypes:
//Top, Push, Pop, isEmpty, isFull
//observer operators
bool isEmpty();
bool isFull();
void Top();
void displayAll(); // goal is to display the stack following the LIFO principle

//transformer operators
void push();
void pop();



int main (){
    int choice;
    while (true){
        std::cout<<"============================\n";
        std::cout<<"Stack operations:"<<std::endl;
        std::cout<<"[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] displayAll"<<std::endl;
        std::cin >> choice;
        std::cout<<"====================\n";
        switch (choice){
            case 1: push();
            break;
            case 2: pop ();
            break;
            case 3: std::cout<<"is the stack empty?"<<isEmpty()<<std::endl;
            break;
            case 4: std::cout<<"is the stack full?"<<isFull()<<std::endl;
            break;
            case 5: Top();
            break;
            case 6: displayAll();
            break;
            default: std::cout<<"Invalid Choice!"<<std::endl;
            break;
        }
    }
    return 0;
}
```

```
49  //function definition
50  bool isEmpty(){
51      return (top == -1);
52  }
53
54  bool isFull(){
55      return (top == maxSize-1);
56  }
57  void Top(){
58      // error catching:
59      if (isEmpty()) return;
60
61      //prints the top element
62      std::cout<<"The top element is:" << stackArr [top] <<std::endl;
63  }
64
65  void push(){
66      //error catching:
67      if (isFull()){
68          std::cout<<"stack overflow!"<<std::endl;
69          return;
70      }
71
72      //ask the user what they want to insert:
73      std::cout<<"What number do you want to push?";
74      std::cin >> newData;
75
76      //storing to the stack:
77      stackArr[++top] = newData;
78  }
79  void pop(){
80      // error catching:
81      if(isEmpty()){
82          std::cout<<"stack underflow!"<<std::endl;
83          return;
84      }
85
86      // display the item to be popped:
87      std::cout<<"Popping:" << stackArr[top] <<std::endl;
88
89      // we remove the element from the stack
90      top--;
91  }
```

```
void displayAll(){
    // error catching:
    if (isEmpty()) return;

    // display all elements from top to bottom:
    std::cout << "Stack from top to bottom: ";
    for (int i = top; i >= 0; i--)
        std::cout << stackArr[i] << " ";

    std::cout << std::endl;
}
```

CODE ANALYSIS

This C++ program implements a custom, array-based stack with a maximum size of 10 (MAXSIZE 10) and a global variable top initialized to -1. It defines helper functions isEmpty() and isFull() to check if the stack is empty or reached its capacity, preventing underflow and overflow errors during operations.

Inside main(), a while loop runs a menu allowing the user to choose stack operations. Selecting push() adds a new element by incrementing top and storing the value in stackArr[top], while pop() displays and removes the top item by decrementing top. The Top() function shows the current top element, and displayAll() prints all elements from top to bottom (i = top down to 0) to follow the Last-In, First-Out (LIFO) order.

ILOB2

MAIN

```
#include <iostream>
#include "stacklist.h"

int main (){

  std::cout<<"---PUSHING 'c' 'p' 'e' ONTO THE STACK ---- \n";
  //push
  push('c');
  push('p');
  push('e');

  //display the current top
  Top<char> (); //Top should be 'e'
  displayALL<char>();
  std::cout<<"\n ------ Poppping all the elements ------ \n";

  while(!isEmpty<char>()){
    std::cout<<"Popped:" << pop<char>() <<std::endl;

  }
  std::cout<<"\n ------ Testing the Underflow ----- \n";
  pop <char>();

  return 0;
}
```

H FILE

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
        T data;       // VALUE THAT WE NEED STORE
        Node* next;  // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

// pop
template<typename T>
T pop(){
    //check if the stack is empty:
    if(head<T> == nullptr){
        std::cout<< "stack overflow. "<<std::endl;
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

    // move the head to the next node
    head<T> = head<T>->next;
    //free the removed node
    delete temp;
    //return the removed value
    return tempVal;
}
```

```
// top
template<typename T>
void Top(){
    // error catching:
    if(isEmpty<T>()){
        std::cout<<"Stack underflow "<<std::endl;
        return;
    }

    // display the Top node
    std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
}

//display All
template <typename T>
void displayALL(){
    if (head<T> == nullptr){
        std::cout << "stack undeflow\n";
        return;
    }

    std::cout<< "Displaying the stack: \n";
    Node<T>* temp = head<T>;
    while(temp != nullptr){
        std::cout << temp->data << "\n";
        temp = temp->next;
    }

    std::cout << "\n";
}

#endif
```

Push
This operation adds a new element onto the top of the stack by creating a new node, assigning it the provided value, and updating the head pointer so that this new node becomes the current top.

Pop
This operation removes and returns the element at the top of the stack. Shifting the head pointer to the next node in line and freeing the memory of the removed node.

isEmpty
This operation checks whether the stack contains any elements by evaluating if the head pointer is currently pointing to nullptr, returning true if empty and false otherwise.

Top
This operation displays the data of the element currently at the top of the stack without removing it, while checking to prevent underflow if the stack is empty.

displayALL
This operation traverses the linked list from the top node down to the end, printing each element's stored value to display the full contents of the stack.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

CODE

```cpp
#include <iostream>
#include <string>
using namespace std;

class ArrayStack {
private:
    char arr[100];
    int top;

public:
    ArrayStack() {
        top = -1;
    }

    void push(char c) {
        arr[++top] = c;
    }

    void pop() {
        if (top != -1)
            top--;
    }

    char peek() {
        return arr[top];
    }

    bool isEmpty() {
        return top == -1;
    }
};

struct Node {
    char data;
    Node* next;
};
```

```
class LinkedStack {
private:
    Node* top;

public:
    LinkedStack() {
        top = NULL;
    }

    void push(char c) {
        Node* temp = new Node;
        temp->data = c;
        temp->next = top;
        top = temp;
    }

    void pop() {
        if (top != NULL) {
            Node* temp = top;
            top = top->next;
            delete temp;
        }
    }

    char peek() {
        return top->data;
    }

    bool isEmpty() {
        return top == NULL;
    }
};

bool match(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
```

```
        (open == '{' && close == '}');
}

bool checkArray(string exp) {
    ArrayStack s;

    for (int i = 0; i < exp.length(); i++) {
        char c = exp[i];

        if (c == '(' || c == '[' || c == '{')
            s.push(c);

        else if (c == ')' || c == ']' || c == '}') {
            if (s.isEmpty())
                return false;

            char open = s.peek();
            s.pop();

            if (!match(open, c))
                return false;
        }
    }

    return s.isEmpty();
}

bool checkLinked(string exp) {
    LinkedStack s;

    for (int i = 0; i < exp.length(); i++) {
        char c = exp[i];

        if (c == '(' || c == '[' || c == '{')
            s.push(c);
```

```
        else if (c == ')' || c == ']' || c == '}') {
            if (s.isEmpty())
                return false;

            char open = s.peek();
            s.pop();

            if (!match(open, c))
                return false;
        }
    }

    return s.isEmpty();
}

int main() {
    string expression;

    cout << "Enter an expression: ";
    getline(cin, expression);

    cout << "\nUsing Stack (Array):\n";
    if (checkArray(expression))
        cout << "Expression is VALID." << endl;
    else
        cout << "Expression is INVALID." << endl;

    cout << "\nUsing Stack (Linked List):\n";
    if (checkLinked(expression))
        cout << "Expression is VALID." << endl;
    else
        cout << "Expression is INVALID." << endl;

    return 0;
}
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter an expression: (A+B)+(C-D) Using Stack (Array): Expression is VALID. Using Stack (Linked List): Expression is VALID.	Valid because all opening symbols have matching close symbols in the correct order, so both stack implementations return valid.

((A+B)+(C-D)	N	Enter an expression: ((A+B)+(C-D) Using Stack (Array): Expression is INVALID. Using Stack (Linked List): Expression is INVALID.	Invalid because one opening parenthesis is left unmatched at the end of the expression, so the stack is not empty.
((A+B)+[C-D])	Y	Enter an expression: ((A+B)+[C-D]) Using Stack (Array): Expression is VALID. Using Stack (Linked List): Expression is VALID.	Valid because every opening symbol has the correct matching closing symbol.
(A+B]+[C-D]}	N	Enter an expression: (A+B]+[C-D]} Using Stack (Array): Expression is INVALID. Using Stack (Linked List): Expression is INVALID.	Invalid because the closing brace does not match the opening bracket, resulting in mismatched symbols.

CONCLUSION

Through this activity, I learned how stacks work using the Last In, First Out (LIFO) principle and how they can be implemented using both arrays and linked lists. I also learned how stacks can be used to check whether symbols in an expression are balanced. While both implementations produce the same results, I understood that arrays are simpler and faster but have a fixed size, whereas linked lists are more flexible because they can grow dynamically. The procedure helped me better understand the push and pop operations and how they are applied in solving real programming problems. Overall, I think I completed the activity successfully because both implementations worked correctly and produced the expected outputs. However, I still need to improve my understanding of linked lists, pointers, and writing efficient and organized code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.